

Компьютерная графика

Точнее, введение в компьютерную графику.

[Из практической информации (особенно по использованию OpenGL) я записывал далеко не всё, так как всё равно есть презентации или документация/руководства, но важная теория записана]

Что почитать

Эйнджел Э. Итеративная графика. Вводный курс на OpenGL

<https://nehe.gamedev.net/>

<https://www.opengl.org/>

1. Лекция 1

Графические ядра являются более продвинутыми библиотеками, занимающимися, в отличие от (OpenGL, Direct3D) вопросами ввода, хранения информации и другими.

Компьютерная графика — это наука, предметом изучения которой является создание, хранение и обработка моделей и из изображений с помощью ЭВМ.

Термины есть в презентации.

Визуализация — процесс преобразования исходной сцены в плоское изображение.

Главная координатная система объекта (локальная система координат) — в ней определяется вершины объекта. Центр этой системы определяется целесообразностью; обычно это геометрический центр модели или угол модели.

Переход из одной координатной системы в другую.

Вершины соединяются рёбрами. Если указаны только вершины и рёбра, то это каркасная модель. Но визуализация каркасной модели создаёт неоднозначность в определении того, какое ребро ближе.

Дальше может быть заданы полигоны (грани). Получается полигональная модель. Она определяет часть пространства, представляющую модель, то есть ограничивают пространство.

Полигональная сетка — наиболее распространённый вид полигональных моделей, аппроксимация с помощью полигонов.

Также есть (CSG) конструктивное твердотельное моделирование: стандартные объекты с задаваемыми свойствами, и модель строится с помощью теоретико-множественных операций (пересечение, объединение) над этими объектами. Модель представляется в виде дерева, где листья — библиотечные объекты, а все остальные узлы — операции над ними.

Если нас интересует не только внешний вид модели, но и внутреннее пространство, то могут использоваться модели на основе октодеревьев. Октодерево (octree) — это восьмеричное дерево, которое строится разрезанием фрагментов пространства тремя плоскостями на 8 частей. На практике чаще используются разряженные октодеревья — разбиваются только неоднородно заполненные фрагменты. Это воксельные модели.

На входе описание модели, на выходе растр — прямоугольная матрица пикселей. Для описания растра заводится буфер цвета, где каждый элемент соответствует пикселю. Размер элемента зависит от используемой палитры и глубины цвета (количество бит на цвет). Для цветных изображений используется RGB. В современных системах буферов больше одного (напр. для пост. обработки).

Современные графические системы реализуют визуализацию с помощью графического конвейера, которому на вход подаются данные модели, а выдаёт он двумерный образ. В конвейере все операции выполняются одновременно.

Большинство команд конвейера — команды его настройки, которая *происходит перед использованием*. То есть порядок этих команд обычно не важен. Но конвейер может перестраиваться между отображением разных моделей.

2. Лекция 2

Конвейер можно представить в виде конечного автомата — присутствует внутреннее состояние.

VRP — Viewer reference point — положение наблюдателя (камеры).

View plane — плоскость наблюдения, на которую происходит проекция.

VPN — нормаль плоскости наблюдения.

Также нужно направление вверх — вектор v .

$v * VPN$ или $VPN * v = u$;

Используется 2 вида систем координат — левосторонняя и правосторонняя. Правосторонняя определяется по правилу правой руки: отогнутый большой палец правой соответствует X, выпрямленный указательный — Y, согнутый средний — Z. Для левосторонней

видовое преобразование переводит из правосторонней системы координат, используемой при моделировании, в левостороннюю видовую систему координат. В видовой системе координат определяется видимый объём (CVV — clip view volume). Этот объём формируется набором из 6 плоскостей — левая, правая, верхняя, нижняя, ближняя, дальняя секущие плоскости. Всё вне видимого объёма отсекается и дальше конвейером не обрабатывается.

Проецирование — преобразование пространства размерности m в пространство меньшей размерности.

Рестеризация — получение всех внутренних точек объекта, исходя из характеристических (управляющих) точек объекта.

3. Аффинные преобразования на плоскости и в пространстве

Масштабирование определяется масштабирующими коэффициентами.

$$S = \begin{bmatrix} S_x & 0 \\ 0 & S_y \end{bmatrix} \text{ — матрица масштабирования}$$

Коэф. S_x — масштабирование по x , S_y — по y . Если больше 1, то увеличивает, меньше — уменьшает. Если отрицательный — отражение относительно оси.

Поворот определяется точкой поворота и углом.

$$R = \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix} \text{ — матрица поворота.}$$

Положительное направление поворота — против часовой стрелки, отрицательное — по часовой.

Для переноса вводятся однородные координаты

Точка на плоскости в однородных координатах определяется тройкой (X, Y, W) , где W — масштабирующий коэффициент, не равный 0. Координаты X и Y могут быть получены из декартовых координат посредством умножения на W . Однородные координаты не обладают однозначностью — одной точке декартовых координат соответствует множество точек однородных координат.

Наибольший интерес представляет частный случай однородных координат, когда $W = 1$. В этом случае они совпадают с декартовыми.

$$T = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ dx & dy & 1 \end{bmatrix} \text{ — матрица переноса.}$$

Скос

Композиция преобразований — матрицы перемножаются в порядке применения.

Преобразования не коммутативны — порядок имеет значение.

Преобразование в пространстве

Теперь матрица 4×4 .

Поворот в пространстве производится относительно вектора (обязательно с направлением; длина не важна, но должна быть больше 0 очевидно).

Положительный поворот в пространстве — поворот против часовой стрелке, глядя с конца вектора, вокруг которого производится поворот.

Для поворота вокруг произвольного вектора мы выполняем поворот вокруг оси X чтобы вектор поворота совпал с плоскостью XOZ , затем поворот вокруг оси Y чтобы вектор поворота совпал с осью Z , затем выполняем поворот на требуемый угол вокруг оси Z , и затем обратные повороты вокруг оси Y и вокруг оси X .

Существуют частные случаи коммутативности преобразований

- перенос с переносом
- масштабирование с масштабированием
- поворот с поворотом вокруг одной и той же оси
- масштабирование с поворотом если масштабирование равномерно ($S_x = S_y = S_z$)

Преобразование можно рассматривать как изменение системы координат.

Hub - OS — Язык высокого уровня — Graphical kernel system или библиотека (OpenGL/Vulkan) - устройство

4. Визуализация двумерных моделей

В простейшем 2D-конвейере используются 3 системы координат:

- WC — мировые координаты — используются на прикладном уровне, для описания модели;
- NDC — нормализованные координаты устройства — независимая от устройства декартова система координат, приведённая к некому фиксированному диапазону (напр. 0-1 или -1..1) ; используется для хранения моделей, обеспечивая защиту от переполнения и потери точности;
- DC — координаты устройства — координатная система, определяемая конкретным устройством вывода графической информации.

Между ними есть преобразования как в одну сторону, так и обратно.

Для проведения преобразований нужно определить правило отображения (преобразования) точки. В основе лежит преобразование из одной прямоугольной области в другую, подобную первой.

Исходный прямоугольник называется окном преобразования (не путать с окном оконной системы). Он определяется парой точек в исходной системе координат.

Второй прямоугольник — поле вывода (viewport). Определяется парой точек в целевой координатной системе.

Преобразование координат из одного прямоугольника в другой согласно пропорции довольно очевидно — линейная интерполяция. Это преобразование можно выразить как смещение с масштабированием, одной матрицей. $Y_v = S_y + Y_w + T_y$

Панорамирование — рассматривание разных частей модели. Реализуется движением окна без изменения его размеров.

Zoom (лупа/масштабирование) — реализуется изменением размера окна без изменения размера viewport'a (с сохранением подобия). Обратный процесс — unzoom.

Алгоритм Сазерленда-Кохена

Алгоритм Сазерленда-Кохена позволяет определить принадлежность примитива прямоугольнику. Эффективен, когда почти все примитивы находятся за пределами прямоугольника или почти все лежат внутри прямоугольника. С помощью этого алгоритма можно эффективно решать две задачи: указание отрезка (pick) и отсечение (clipping). В алгоритме через стороны исходного прямоугольника проводятся прямые, которые разбивают плоскость на 9 зон. Каждая из зон получает 4-битный код: единица в 0-м бите означает, что зона лежит выше целевого прямоугольника, в 1-м — ниже, во 2-м — правее, в 3-м — левее (порядок не обязательно такой). Точка отрезка, попадая в одну из этих зон, наследует код этой зоны. Если код₁ **and** код₂ ≠ 0, то отрезок лежит вне целевого прямоугольника. Если код₁ **and** код₂ = 0, то отрезок целиком лежит внутри прямоугольника. В других случаях требуется дополнительное исследование. В этом случае исследуемый отрезок пересекают с границами окна, начиная с 1-й точки. Для полученной точки пересечения формируют новый код, и проводят новый тест. В худшем случае нужны 4 пересечения.

Точки пересечения рассчитываются тривиально, так как у вертикальных линий заведомо известен X-координата пересечения, а у горизонтальных — Y.

5. Проекция

В переводе пространственных объектов на плоскость несоответствие размерности устраняется путём введения проекции. В общем случае, проекция преобразует точки одной координатной системы в координатную систему меньшей размерности.

Проекция 3D объекта представлена в виде множества точек, строящихся при помощи множества прямых линий проекций, которые выходят из центра проекции, проходят через каждую точку объекта и, пересекая картинную плоскость образуют картинную проекцию называются плоской геометрической проекцией.

Плоские геометрические проекции можно разделить на 2 класса: центральные и параллельные. Различия между ними определяются расстоянием центра проекции от проекционной плоскости: у центральной конечно, у параллельной — бесконечно.

Для параллельных вместо центра проекции вводят понятие направления проецирования.

В ходе проективных преобразований, проецируемые объекты могут изменять свою форму. Это называется укорачиванием. Оно характеризуется коэффициентом укорачивания. Если проецируется отрезок, то коэффициент определяется как отношение длины проекции отрезка к длине самого отрезка.

При проецировании отрезка параллельным проецированием, величина коэффициента зависит только от угла проецируемого отрезка к проекционной плоскости.

При центральном проецировании также имеет место укорачивание, но оно отличается свойствами от параллельного и называется перспективным укорачиванием. Оно уже зависит и от угла, и от расстояния до проекционной плоскости.

В общем случае, при перспективным проектировании объекты будут иметь одинаковые коэффициент укорачивания, только если они лежат в плоскости, параллельной плоскости проецирования.

Основные понятия, определяющие проекцию:

1. направления проецирования или центр проецирования (положение наблюдателя),
2. расположение проекционной плоскости,
3. положение главной координатной системы проецируемого объекта (т. е. расположение проецируемого объекта).

Если направление проецирования совпадает с нормалью (ортом) плоскостью проецирования, то проекция ортографическая, иначе — косоугольная.

Наиболее широко используемые ортографические — вид спереди (фронтальная), сбоку и сверху (план).

Для получения вида спереди необходимо расположить объект так, чтобы ось Z главной координатной системы объекта (ГКО) была перпендикулярна проекционной плоскости. Для получения плана — ось Y должна быть перпендикулярна проекционной плоскости, вид сбоку — X (но на практике это зависит от самого объекта, как он расположен в своей системе координат).

Простые ортографические проекции получили широкое распространение в связи с тем, что, измерение линейных размеров можно производить с точностью до некоторого коэффициента прямо на проекции. И искажение углов в плоскости, параллельной проекционной, не происходит.

По этим проекциям представить реальный вид объекта трудно. Этого недостатка нет в аксонометрических проекциях: триметрия, диметрия, изометрия. В этом случае оси не перпендикулярны плоскости проекции, так что видно сразу несколько сторон объекта. При этом, сохраняется параллельность прямых, а углы и расстояния можно измерять вдоль каждой оси отдельно со своим коэффициентом укорачивания.

При изометрической проекции, коэффициенты укорачивания вдоль всех трёх осей одинаковы (углы между главным координатными осями и проекционной плоскостью равны); при диметрической — у любых двух осей равны, при триметрии — все разные.

Косоугольные проекции сочетают в себе свойства ортографических со свойствами аксонометрии: проекционная плоскость перпендикулярна одной из главных координатных осей, поэтому одна сторона объекта параллельна плоскости проецирования (нет укорачивания), а проецирование других позволяют измерять длины (но не углы) со своими коэффициентами укорачивания.

Существует 2 наиболее популярные косоугольные проекции: Кабине и Кавалье. В проекции Кабине угол между плоскостью и направлением проецирования — 45° . В проекции Кавалье — $\arcsin(1/2)$, что даёт коэффициент укорачивания 0.5 для оси, перпендикулярной плоскости, параллельной плоскости проецирования.

Одним углом наклона направления проектирования нельзя определить направление в пространстве. Так что определяется ещё угол α — угол между проекцией оси X и проекцией направления проецирования.

Центральные проекции любой совокупности параллельных прямых, которые не параллельны проекционной плоскости, будут сходиться в точке схода.

Точка схода — это точка на проекционной прямой, в которой сходятся центральные проекции параллельных прямых, не параллельных проекционной плоскости.

Точек схода может быть бесконечное количество, но главных 3. Главные определяются по осям главной координатной системы (линии, параллельные осям). Перспективные (центральные) проекции делятся по количеству главных точек схода. Их количество зависит от количества ГКО, не параллельных проекционной плоскости: две параллельно — одна главная точка схода, одна параллельна — 2 точки, никакая не параллельна — 3 точки.

6. OpenGL

Команды реализуются в виде функций:

`glCommandName[1 2 3 4][b s i f d ub us ui][v]`

1 2 3 4 — количество параметров,

b s i f d ub us ui — типы параметров,

v — обозначает, что аргумент — указатель на массив. Количество и тип элементов массива определяется другими символами.

Константы в формате GL_НАЗВАНИЕ_КОНСТАНТЫ.

Если команды принадлежат расширениям OpenGL, то префикс совпадает с названием библиотеки.

OpenGL реализован на основе конвейера визуализации, который удобно представить как конечный автомат, для которого существует множество возможных состояний, некоторое начальное состояние, некая функция выхода (реакции или вывода результата).

Состояния конвейера определяются совокупностью значений *переменных состояния*. Для каждой переменной состояния определено начальное состояние, и как следствие, определено начальное состояние конвейера. Это переменные могут быть даже векторами и массивами.

Для переменных состояния существуют команды, которые позволяют опросить их состояние: `glGetBoolean()`, `glGetFloatv()`, `glGetError()`, ...

Есть стек атрибутов.

Внутри конвейера OpenGL используется 4 вида матриц: видового преобразования (`GL_MODELVIEW`), проецирования (`GL_PROJECTION`), текстуры, цвета. По умолчанию все матрицы единичные.

С каждым типом матрицы связан свой стек. Глубина стека зависит от типа матрицы.

Механизм доступа к матрицам и их стекам единый — команда одна и та же на всех, нужно переключать текущий выбранный тип с помощью `glMatrixMode()`.

Некоторые команды неявно изменяют текущий выбранный тип.

`glLoadIdentity()` - заменяет текущую матрицу (ту, которая на вершине стека) единичной.

`glPushMatrix()`, `glPopMatrix()`, `glMultMatrix()`.

glScale(sx, sy, sz) формирует матрицу (4×4) масштабирования, затем умножает текущую на сформированную и заменяет результатом текущую (т. е. Производит композицию с текущей матрицей). Аналогично с glRotate(angle, x, y, z) и glTranslate(x, y, z) аналогично.

Задание векторных примитивов

Их можно задавать разными методами. Точки, линии, многоугольники. Они описываются совокупностью координат. Каждый геометрический объект описывается как упорядоченный набор вершин. Набор вершин ограничивается операторными скобками glBegin(), glEnd().

Многоугольники должны быть плоскими и выпуклыми.

Внутри glBegin...glEnd можно задавать атрибуты (цвет, нормали, текстурные координаты, ...).

7. Трёхмерный видовой конвейер

Память приложения

Поверхностные операции

Все геометрические объекты описываются своими вершинами или контрольными точками. Так описываются параметрические кривые и плоскости. Вычислитель — метод, который генерирует координаты вершин, нормали, координаты текстур и цвета точек на основе контрольных точек.

Фрагмент можно рассматривать как прообраз пикселя. При операциях с ними определяются тексели текстуры, соответствующей фрагменту. Над фрагментами проводится серия тестов, от результатов которых зависит, попадёт ли информация о фрагменте в буфер кадра.

Отдельно на конвейере может обрабатываться пиксельная информация. Эти данные распаковываются с учётом форматов. Результат либо записывается в память текстуры, либо отправляется на растеризацию.

Тесты, которые проводятся над фрагментами:

1. Scissor (ножницы) — тест отсечения — на экране определяется прямоугольная область, и тест пропускает только фрагменты, которые попадают внутрь этого прямоугольника.
2. Альфа-тест — производит отбор компонентов на основе альфа-канала (прозрачности) — задаётся пороговое значение и функция сравнения.
3. Stencil-тест — тест трафарета — используя трафаретный буфер, которая действует как универсальная маска для всего экрана — для каждого пикселя в трафаретном буфере хранится целое число, которое сравнивается с заданным эталоном, используя битовую маску, и, в зависимости от результата сравнения, можно управлять или содержимым буфера, или видимостью пикселей. Также используется для создания эффекта зеркала, создания теней, штриховки и т. д.
4. Тест глубины (depth-test) -

Шейдер — это подпрограмма, выполняемая на стороне графического устройства в составе конвейера. Для OpenGL шейдеры пишутся на GLSL.

Обычно выделяют вершинные шейдеры (вызываются для каждой вершины), геометрические (способны обработать не только отдельные вершины, но и весь примитив целиком) и фрагментные (связаны с обработкой фрагментов, текстурированием и т. п.).

8. Проекция 2

Матрица ортогографического проецирования тривиальна — она просто делает нулевой ту координату, которая соответствует оси, перпендикулярной плоскости проекции.

9. Реализация проекций в OpenGL

`glOrtho()` — создание матрицы ортогональной проекции, видовой объём — параллелепипед.

В видовых координатах направление оси Z обратно мировым координатам.

Направление проецирования параллельно оси Z.

Всегда должно быть $near < far$.

В матрице ортогональной проекции 3-й элемент на главной диагонали отрицательный чтобы инвертировать ось Z.

Масштабирование и сдвиг — для перехода от заданного объёма к каноническому.

Операция отсечения относительно произвольного объёма трудоёмка, а относительно канонического — эффективнее.

`glFrustum()` - центральная проекция, видимый объём — усечённая четырёхгранная пирамида.

Центр проекции расположен в точке (0; 0; 0) видовой системы координат. Проекционная плоскость совмещена с передней секущей ($near$). Величины $near$ и far могут быть только положительны, $far > near$.

При использовании центральной проекции почти всегда приходится сдвигать модель в сторону $-Z$ мировой системы координат чтобы она попала внутрь видимого объёма.

`gluPerspective()` Aspect ratio = W/H

`gluLookAt()`

10. Освещение и свет

Физическая природа света очень сложна и расчёт её модели очень затратно. Поэтому в компьютерной графике используются базовые понятия теории света. Свет, падающий на поверхность объекта, может быть поглощён, отражён или пропущен. Объект виден при пропуске (преломлении) или отражении света. Если объект поглощает весь свет, то он не виден и называется абсолютно чёрным телом.

Если объект отражает весь падающий свет, он называется идеально отражающим — зеркалом. Количество отражённой, поглощённой или пропущенного света зависит от длины волны падающего света и свойств поверхности освещаемого объекта.

Свойства отражённого света зависят от направления и строения источника света. Отражённый свет может быть диффузным или зеркальным.

Существующие модели освещения можно разделить на

- глобальные — при расчёте освещённости учитывается как прямо падающий свет от источника, так и отражённый и преломлённый другими объектами свет ; дают хорошее качество, но требуют больших вычислительных затрат, так что используются для статических сцен (и CGI)
- локальные — в расчётах опираются только на свет, идущий прямо от источника, учитывается направление на источник, расстояние до него и положение объекта в пространстве и отражательные свойства его поверхности, а влияние других объектов исключено, теней и зеркал нет, так что реалистичность очень низкая.

Расчёт освещения обычно производится в нескольких характеристических точках объекта, а интенсивность освещения остальных точек рассчитывается с помощью интерполяции.

Рассмотрим локальную модель освещения «простая модель освещения».

Диффузное отражение происходит за счёт шероховатости поверхности объекта. Лучи, отражённые идеальным диффузным отражателем, отражаются равномерно во всех направлениях. Вектор L — на источник света, n — нормаль поверхности.

Говорят, что свет как бы проникает под поверхность объекта, поглощается и повторно излучается. Положение наблюдателя, до определённых углов, не имеет значения.

Свет от точечного источника отражается по закону косинуса Ламберта:

$$I_d = I_l K_d \cos \theta$$

K_d — коэф диффузного отражения поверхности

I_l — интенсивность источника

θ — угол падения света. $0 < \theta < \pi/2$

Если рассматривать только диффузное освещение, то для нас освещение двух точек, расположенных на разном расстоянии от источника, будет неотличимо. В реальном мире существует затухание света, квадратичное от расстояния.

Второй момент, который нужно учесть: существуют точки, на которые прямой свет от источника не падает.

$$I_d = I_a K_a + I_l K_d \cos \theta / (d+K)$$

I_a — рассеянного источника

K_a — коэф отражения света, испускаемого рассеянного источника

$0 < \text{Коэф-ы} < 1$

Величина ослабления зависит от расстояния до источника.

Диффузная составляющая — основа освещения.

В моделях, которые учитывается только диффузное освещение, объекты равномерно матовые. Большую реалистичность даёт зеркальная составляющая, которая дополняет картину бликами. Зеркальная составляющая, в отличие от диффузной, зависит от положения наблюдателя.

В силу большой сложности физической сложности зеркального отражения, в простых моделях пользуются моделью Буи Фонга:

$$I_s = I_l w(\theta, \lambda) \cos^n \alpha$$

$w(\theta, \lambda)$ - кривая отражения, представляющая отражение зеркально отражённого света к падающему как функцию

$\cos^n \alpha$ — аппроксимация пространственного распределения отражённого света

Функцию w обычно заменяют подобранной константой.

Все 4 вектора (L , R , S , n) всегда лежат в одной плоскости.

11. Плоское закрашивание

Для расчёта интенсивности в одной точке нужно рассчитать те 4 вектора. Это достаточно нетривиальная задача.

Нормаль плоскости рассчитывается тривиально — векторное произведение двух любых неколлинеарных векторов в этой плоскости, обычно вектора, получающиеся из вершин грани.

Если для всех точек плоского многоугольника мы примем все вектора одинаковыми, то получим плоское закрашивание (затенение).

На стыках многоугольников, имеющих разные нормали, возникает резкий переход интенсивности и из-за особенности зрительной системы человека, очень чувствительной к этому, при резком переходе от светлого к тёмному мы видим резкие полосы — полосы Маха.

Для борьбы с полосами Маха используют различные методы. Среди них метод закрашивания Гуро (гладкое затенение). Этот метод предполагает усреднение нормалей и билинейную интерполяцию интенсивностей рассчитанных, для каждой вершины многоугольника. Для усреднения нормали в некоторой вершине усредняются все нормали граней, стыкующихся данной вершиной (результат нормализованный — вектор единичной длины). Сложность сводится к разработке модели отражающей топологии объекта — нужно искать для каждой вершины все стыкующиеся многоугольники.

Для проведения билинейной интерполяции рассчитываются интенс

Для определения интенсивности в произвольной точке P производится линейная интерполяция интенсивностей в точках Q и R — точках пересечения контура многоугольника со сканирующей строкой (с плоскостью сканирующей строки).

В силу линейности интерполяции можно получить инкрементальную версию алгоритма — каждое новое значение рассчитывается на основе предыдущего.

Закраска Гуро даёт более реалистичное представление поверхности, однако не устраняет полностью полос Маха, так как на стыках многоугольников обеспечивается только непрерывность значения, а не непрерывность его изменения (первой производной). Вторая проблема связана с тем, что усреднение нормалей может давать одинаковый результат для различно ориентированных многоугольников.

Закраска Фонга менее подвержена появлению полос Маха, так как при ней интерполируются не величины интенсивности, а вектора нормалей. Вектора интерполируются посредством интерполяции их начальных и конечных точек, а каждая точка интерполируется покомпонентно.

12. Реализация освещения в OpenGL

В OpenGL все источники света точечные (испускает во все стороны равномерно). Существует 2 типа точечных источников — направленные и позиционные.

Направленным считается источник, расположенный в бесконечности (бесконечно далеко, напр. Солнце). Для таких источников характерно только направление и интенсивность — все лучи параллельны. Для направленных источников затухание отсутствует.

Позиционный источник расположен в непосредственной близости от освещаемой сцены — у него есть позиция в мировых координатах. Позиционный источник с конусом-ограничителем — прожектор.

Модель света считается отдельно для красного, зелёного и синего компонента (и свойства материала задаются тоже по ним).

Интенсивность задаётся 12 числами (в теории 9)

Свойства источников света задаётся командой `glLight`(имя источника, имя свойства, значение).

`GL_SPOT_EXPONENT` — фокусирование света внутри конуса

`GL_SPOT_CUTOFF` — угол раскрытия светового конуса (угол между центральной осью и границей, т. е. 180 — световой конус отсутствует)

`GL_AMBIENT` (рассеянный), `GL_DIFFUSE` (диффузный), `GL_SPECULAR` (зеркальная) интенсивности — четвёрки (R; G; B; A), в диапазоне [0; 1].

`GL_POSITION` — комбинированное свойство, которое определяет тип источника света и, в зависимости от заданного типа, либо позицию (для позиционного), либо направление освещения (точку, из которой можно провести прямую в начало координат и получить направление; для направленного). Значение определяется четвёркой (x;y;z;w), где w определяет тип источника (1 — позиционный, 0 — направленный). В любом случае заданная четвёрка умножается на матрицу модельно-видового преобразования.

`GL_SPOT_DIRECTION` — направление светового конуса.

`GL_LIGHT_MODEL_LOCAL_VIEWER` — будет ли учитываться реальное направление наблюдателя или оно считается параллельной оси Z.

Все плоские многоугольники имеют лицевую и изнаночную сторону. Они определяются по направлению обхода вершин после проецирования — в положительном (против часовой стрелки) или отрицательном (по часовой стрелке). Направление обхода для лицевой грани определяется `glFrontFace()`: `GL_CW`, `GL_CCW`.

Предусмотрен механизм отбора граней по типу — параметр `GL_CULL_FACE`, команда `glCullFace()`: `GL_FRONT`, `GL_BACK`.

В OpenGL используется множество способов задания параметров материалов.

```
glMaterial(  
    GL_FRONT|GL_BACK,  
    GL_AMBIENT|...,  
).
```

`GL_SHININESS` — распределение (угол) зеркально отражённого света после отражения (0 — абсолютно шероховато, 128 — зеркало)

`GL_EMISSION` — интенсивность собственного испускаемого света.

13. Цвет в компьютерной графике

Свет бывает

1. ахроматическим (монохромный) — характеризуется только интенсивностью
2. хроматический — содержит длины (электромагнитных) волн в произвольных неравных количествах. Цвет описывают, используя 3 величины:
 1. цветовой тон — положение в спектре,
 2. насыщенность — чистота (степень ослабления/разбавления данного цвета белым),
 3. светлоту — отражает представление об интенсивности как о факторе, независимом от цветового тона и насыщенности. При светоиспускании (а не отражении) используется термин «яркость».

Для описания цвета используются цветовые модели — математические способы определения цветовых областей спектра.

Любая модель должна удовлетворять требованиям:

1. реализовывать определение цвета некоторым стандартным способом, не зависящим от конкретного устройства
2. точно задавать диапазон воспроизводимых цветов
3. учитывать механизм восприятия света

Большинство применяемых моделей можно разбить на 3 класса:

1. аддитивные (напр. RGB)
2. субтрактивные (напр. CMY)

3. перцепционные (напр. HSV)

Каждая модель характеризуется цветовым охватом — диапазоном цветов, который может быть описан данной моделью. Он же характеризует устройства.

На сегодняшний день большую популярность получили трёхкомпонентные модели основанные на том, что в центральной части сетчатки глаза [человека] находится 3 вида колбочек, воспринимающие красный, зелёный и синий.

В (аддитивных) моделях основным называется цвет, который не может быть получен линейной комбинацией двух других.

Цветовое пространство модели RGB трёхмерно, его удобно представлять в виде куба. Главная диагональ куба — ахроматическая ось ((0;0;0) — чёрный, (1;1;1) — белый).

Ключевой недостаток RGB — очень низкий цветовой охват. Другой — неинтуитивное кодирование.

Субтрактивная модель CMY применяется для отражающих поверхностей — цвета получаются вычитанием из белого. В реальной жизни применения не находит. Поэтому в модель вводят 4-ю компоненту — чёрный, получается CMYK.

Недостатки модели — узкое цветовое пространство (даже меньше, чем у RGB), очень сильная аппаратная зависимость (зависимость от цвета подложки (бумаги)).

CMY тривиально преобразуется в RGB — простым вычитанием из единицы: $(R;G;B) = (1;1;1) - (C;M;Y)$.

Перцепционные модели используют принципы формирования цвета, интуитивно понятные человеку — опираясь на понятия разбела, оттенка и тона.

Разбел получается при добавления белого пигмента к чистому — уменьшается насыщенность.

В основу HSV положены понятия разбела, оттенка и тона. Цветовое пространство определяется шестигранным конусом. Ось V — светлота, S — насыщенность, угол H — тон.

Модель CIE Lab — модель стандартизированного цветового пространства. Модель равноконтрастная — с равным расстоянием между цветами в любых частях цветовой модели.

Цветовое пространство Lab охватывает все различимые человеком цвета.

Цвет определяется 3 компонентами: яркость (lightness; 0 — чёрный, 100 — белый), ось a (от зелёного к красному (-120..+120)) и b (от синего до жёлтого (-120..+120)). Цветовое пространство получается полусферой.

14. Текстуры

Текстуры можно рассматривать как прямоугольные массивы данных, хранящие информацию о цвете. Массив данных называют картой текстур, а элемент массива — текселем. Массивы могут быть 1-, 2-, 3-, 4-мерными.

Для того чтобы наложить текстуру нужно:

1. подготовить данные для текстуры

2. создать текстурный объект
3. загрузить данные в текстурный объект
4. установить фильтры
5. определить метод выхода за диапазон
6. задать функцию наложения текстуры
7. выполнить привязку вершин объекта к текстуре
8. разрешить использование текстуры.

Загрузка текстурной карты — `glTexImage()`.

Фильтры текстуры и поведение текстуры при выходе за диапазоны (для каждой оси отдельно) определяются командой `glTexParameter[f,i][v]()`.

Фильтрация обязательно задаётся для уменьшения и увеличения текстуры.

При увеличении всегда используется текстура 0-го уровня.

`GL_(1)_MIPMAP_(2)` - фильтрация происходит не на 0-м уровне, а на автоматически выбранном: (1) — способ фильтрации, (2) — способ выбора уровня (интерполяции между 2 выбранными уровнями).

`GL_NEAREST` — качество выше, скорость ниже → `GL_LINEAR_MIPMAP_LINEAR`

Текстурная функция (режим наложения текстуры) задаётся командой `glTexEnv[i,f][v]()`.

Привязка текстурных координат к вершинам может производиться вручную (`glTexCoord()`) или автоматически (`glEnable(GL_TEXTURE_GEN_[S,T,R,Q]) + glTexGen()`).

`glTexGen: GL_TEXTURE_GEN_MODE (GL_OBJECT_PLANE, GL_EYE_PLANE);
GL_OBJECT_LINEAR, GL_EYE_LINEAR.`

Если мы хотим повысить эффективность наложения текстур, мы можем заранее подготовить несколько текстурных объектов, загрузить в них текстурные карты, а потом, при визуализации, переключаться между текстурными объектами.

Переключение текстурного объекта — `glBindTexture()`. При первом вызове создаётся текстурный объект. Именами текстуры могут быть целые числа. Получить набор незанятых имён можно с помощью команды `glGenTextures()`. 0-я текстура изначально текущая.

15. Удаление невидимых линий и поверхностей

Метод плавающего горизонта используется для математических поверхностей. Исходная поверхность разбивается пакетом параллельных плоскостей с некоторым шагом. Обычно эти плоскости перпендикулярны одной из базовых осей. В результате получаются кривые. Затем они проецируются на плоскость. Точка видна, когда она на сечении выше, чем на предыдущем сечении.

Через буфер глубины. Изначально он заполняется минимальным значением, которое может появиться при визуализации сцены. Для каждого пикселя после растеризации определяется глубина.

Обратная трассировка лучей.

Вопросы для самостоятельного изучения: устройство (принцип построения и работы) современных мониторов, принтеров, мышек.

Цитаты

«Я приму любой вариант, если вы дойдёте до конца»

«Я предлагаю вам метод грубой силы»

«Предлагаю вам использовать самые примитивные примитивы»

«А то на третьем курсе некоторые уже не помнят, что такое таблица истинности»

«Берём другую ось, докручиваем.»